

Introduction à l'algorithmique et à la programmation Les procédures et fonctions

Les procédures et fonctions

Buts :

- O Savoir expliquer comment faire un travail sans la moindre ambiguïté
- O Les instructions des algorithmes (et des programmes) peuvent vite être nombreuses
- O Il y a souvent des instructions identiques ou très presque identiques
- O Pour simplifier : Définir de nouvelles instructions qui en regroupent plusieurs

Les procédures et fonctions

Exemple 1 : première question (réponse oui / non)

programme Question1 **c'est**

début

rep : caractère;

ecrireEcran("Etes-vous étudiant ? (O/N) :");

lireClavier(rep);

tant que ((rep!='O') ET (rep!='N')) **faire**

ecrireEcran("Entrée erronée, recommencez :");

lireClavier(rep);

finfaire

fin

Les procédures et fonctions

Exemple 2 : deuxième question (réponse oui / non)

programme Question2 **c'est**

début

rep : caractère;

ecrireEcran("Voulez-vous du thé ? (O/N) :"); // la seule différence !!!

lireClavier(rep);

tant que ((rep!='O') ET (rep!='N')) **faire**

ecrireEcran("Entrée erronée, recommencez :");

lireClavier(rep);

finfaire

fin

Les procédures et fonctions

Exemple 3 : plusieurs question (réponse oui / non) **!!! Duplication de code !!!**

programme Questions **c'est**

début

rep : caractère;

ecrireEcran("Etes-vous étudiant ? (O/N) :");

lireClavier(rep);

tant que ((rep!='O') ET (rep!='N')) **faire**

 ecrireEcran("Entrée erronée, recommencez :");

 lireClavier(rep);

finfaire

ecrireEcran("Etes-vous adulte ? (O/N) :");

lireClavier(rep);

tant que ((rep!='O') ET (rep!='N')) **faire**

 ecrireEcran("Entrée erronée, recommencez :");

 lireClavier(rep);

finfaire

ecrireEcran("Voulez-vous du thé ? (O/N) :");

lireClavier(rep);

tant que ((rep!='O') ET (rep!='N')) **faire**

 ecrireEcran("Entrée erronée, recommencez :");

 lireClavier(rep);

finfaire

fin

Les procédures et fonctions

Buts des procédures et fonctions :

- O ré-utilisabilité : aptitude d'un algorithme à être réutilisé pour résoudre des tâches équivalentes à celles pour laquelle il a été conçu
- O Pas de duplication de code (... moins de sources d'erreurs...)
- O Amélioration la lisibilité du programme
- O Simplification le code : en allégeant le code

Les procédures et fonctions

La démarche :

- Rassembler et isoler des instructions
- **Encapsuler** ces nouvelles instructions
- Nommer ce rassemblement

Remarques :

- Programme **modulaire** : une seule modification au bon endroit pour modifier l'ensemble du programme
- Programme : **procédure principale**
- Les fonctions et les procédures s'écrivent en dehors de la procédure principale

Les procédures et fonctions

Plan :

- O Les procédures
 - Procédure sans paramètre
 - Procédure avec paramètres
- O Les fonctions (qui retournent des résultats)
- O Les fonctions récursives

Les procédures et fonctions

Les procédures sans paramètre : le pseudo-code

- O Une procédure : un sous programme indépendant pour capitaliser une suite d'instructions

programme Premier **c'est**

// définition de la procédure

procedure <nomDeProcédure>() **c'est**

debut

 <déclaration des variables de la procédure>

 <Instructions de la procédure>

fin

// définition du programme principal

debut

 <liste d'instructions avec appel à la procédure>

<nomDeProcédure>(); // appel de la procédure

fin

Les procédures et fonctions

Exemple : saisir 10 ages et préciser si la personne est majeure ou non

programme Premier **c'est**

procedure traiterAge() **c'est** // **procédure** totalement indépendante du programme principal

debut

age : entier;

ecrireEcran("votre âge : ");

lireClavier(age); // l'utilisateur donne un âge

si (age >= 18) **alors**

ecrireEcran("Vous êtes majeur ");

sinon

ecrireEcran("Vous êtes mineur ");

finsi

fin

// **programme principal**

debut

nbFois : entier;

nbFois:=1;

tant que (nbFois <= 10) **faire**

ecrireEcran("Donnez ");

traiterAge(); // **Appel de la procédure**

nbFois := nbFois + 1;

finfaire

fin

Les procédures et fonctions

Les procédures avec paramètres :

- paramètre (ou argument) : permet de préciser une ou plusieurs données nécessaires à l'exécution de la procédure.
- chaque paramètre possède un **nom** et un **type** : paramètre formel
- 3 modes de transmission des paramètres
 - **en entrée** : la valeur du paramètre formel est utilisée dans la procédure (**entF**)
 - **en sortie** : permet de transmettre à la procédure appelante (ex. programme principale) la valeur du paramètre (calculé par la procédure) (**sortF**)
 - **en entrée/sortie** : la valeur du paramètre peut être utilisée dans la procédure, modifiée et transmise (**entF/sortF**)

Les procédures et fonctions

Les procédures avec paramètres :

```
// Déclaration de la procédure :  
procedure <nomDeProcedure>(  
    <modeTrans> <nomPara1>:<type1>,  
    <modeTrans> <nomPara2>:<type2> )c'est  
    debut  
        <déclaration des variables de la procédure>  
        <Instructions de la procédure>  
    fin
```

- O <modeTrans> est soit **entF**, soit **sortF**, soit **entF/sortF**
- O <nomPara1> et <nomPara2> sont ensuite considérées comme des variables dans la procédure

Les procédures et fonctions

Les procédures avec paramètres : Appel de la procédure

- Lors de l'appel de la procédure, préciser les valeurs des paramètres
- Ce sont les **paramètres effectifs**, qui seront utilisés *effectivement* dans la procédure
- Un paramètre effectif en entrée (**entE**) peut être :
 - Une constante
 - Une variable (dans ce cas, c'est sa valeur qui est transmise à la procédure)
 - Une expression ((ex : $f(i+1)$ ou $f(8+3)$) (*à éviter dans un premier temps*))
- Un paramètre effectif en entrée/sortie (**entE/sortE**) ou en sortie (**sortE**) ne peut être qu'une variable (*une constante ne pourrait être modifiée*).
- Lors de l'appel, le paramètre formel est remplacé par le paramètre effectif

Les procédures et fonctions

Les procédures avec paramètres : un peu de méthode

- Définir les effets d'une procédure en spécifiant :
 - Le nom de la procédure (nom évocateur)
 - La liste le type et la nature de chaque paramètre
 - param1** de type **réel** en **entrée**,
 - parm2** de type **entier** en **entrée**,
 - param3** en de **type** caractère en **entrée/sortie**
 - ...
- Précondition : décrire l'état des paramètres avant l'appel de la procédure
- Postcondition : décrire l'état des paramètres après l'appel de la procédure

Remarque :

- Il peut ne pas y avoir de paramètres d'entrée **ou** de sortie

Les procédures et fonctions

Exemple de procédure avec paramètres : une procédure pour répondre à une question O/N

```
programme questions c'est
// Definition de la procédure
procedure question(entF msg : chaine(200), sortF rep : caractère) c'est
  debut
    ecrireEcran(msg);
    lireClavier(rep);
    tant que ((rep!='O') ET (rep!='N')) faire
      ecrireEcran("Entrée erronée, recommencez :");
      lireClavier(rep);
    finfaire
  fin
debut
  reponse : caractère;
  message : chaine(200)
  message = "Etes-vous étudiant ? (O/N) : "
  question(entE message, sortE reponse); // Appel de la procédure
  si ((reponse=='O') alors
    message = "Etes vous adulte ? (O/N) : "
    question(entE message, sortE reponse); // Appel de la procédure
    si ((reponse=='O') alors
      message = "Voulez-vous du thé ? (O/N) : "
      question(entE message, sortE reponse); // Appel de la procédure
      si (reponse=='O') alors
        ecrireEcran("Bienvenue !, voici votre thé");
      finsi
    finsi
  finsi
fin
```

Les procédures et fonctions

Exemple de procédure avec paramètres : moyenne de 2 valeurs entières

Programme moyenne c'est

// Definition de la procédure

procedure moyenne(**entF** v1 : entier, **entF** v2 : entier, **sortF** moy : réel) **c'est**

debut

moy:= (v1+v2)/2.0;

fin

debut

// Declaration des variables

n1, n2 : entier;

res : réel;

// Saisie des données

ecrireEcran("Entrez un nombre");

lireClavier(n1);

ecrireEcran("Entrez un nombre");

lireClavier(n2);

// Traitement

moyenne(**entE** n1, **entE** n2, **sortE** res); **// appel de la procédure**

// Affichage

ecrireEcran("La moyenne de ",n1," et ",n2," est ",res);

fin

Les procédures et fonctions

Manipulation des variables dans une procédure : **!!!Les erreurs !!!!**

```
procedure moyenne(entF v1 : entier, entF v2 : entier, sortF moy : réel) c'est  
  debut  
    res : réel; // variable locale à la procédure  
    res:=moy; // moy est un paramètre de sortie (1)  
    v1:= v1+v2; // v1 est un paramètre d'entrée (2)  
  fin
```

(1) Paramètre en sortie : **ne peut être qu'affecté** (tjs à gauche d'une affectation)

(2) Paramètre en entrée : **ne peut pas être affecté** (jamais en partie gauche d'une affectation)

Remarque

O Paramètre en entrée/sortie : **peut être affecté** (en partie **gauche** d'une affectation) **et peut être utilisé pour une affectation** (à **droite** d'une affectation)

Les procédures et fonctions

Les variables dans les procédures

- Les paramètres d'une procédures sont locales à la procédure
- Les variables déclarées dans une procédure sont locales à la procédure (pas utilisables en dehors de la procédure)
- Les constantes déclarées dans une procédure sont locales à la procédure (ne sont pas utilisables en dehors de la procédure)
- Les constantes déclarées en début de programme (en dehors de toutes procédures) sont valables dans toutes les procédures

Les procédures et fonctions

Les variables dans les procédures : Constante déclarée dans la procédure

```
programme moy2 c'est  
procedure moyenne(entF v1 : entier, entF v2 : entier, sortF moy : réel) c'est  
    constante réel NB:=2.0;           // une constante locale à la procédure  
    debut  
        moy:= (v1+v2)/NB;  
    fin
```

```
debut  
    n1, n2 : entier;  
    res : réel;  
    ecrireEcran("Entrez un nombre");  
    lireClavier(n1);  
    ecrireEcran("Entrez un nombre");  
    lireClavier(n2);  
    moyenne(entE n1, entE n2, sortE res);  
    ecrireEcran("La moyenne de ",n1," et ",n2," est ",res);  
fin
```

Les procédures et fonctions

Les variables dans les procédures : Constante déclarée à l'extérieur de la procédure

```
programme moy2 c'est
```

```
constante réel NB:=2.0; // une constante accessibles par toutes les fonctions et procédures
```

```
procedure moyenne(entF v1 : entier, entF v2 : entier, sortF moy : réel) c'est
```

```
  debut
```

```
    moy:= (v1+v2)/NB;
```

```
  fin
```

```
debut
```

```
  n1, n2 : entier;
```

```
  res : réel;
```

```
  ecrireEcran("Entrez un nombre");
```

```
  lireClavier(n1);
```

```
  ecrireEcran("Entrez un nombre");
```

```
  lireClavier(n2);
```

```
  moyenne(entE n1, entE n2, sortE res);
```

```
  ecrireEcran("La moyenne de ",n1," et ",n2," est ",res);
```

```
fin
```

Les procédures et fonctions

La portée des variables : **!!! Les erreurs !!!**

```
programme moy2 c'est  
procedure moyenne(entF v1 : entier, entF v2 : entier, sortF moy : réel) c'est  
debut  
    moy := (n1+n2)/2.0; //n1 et n2 ne sont pas connues à ce niveau  
fin  
debut  
    n1, n2 : entier;  
    res : réel;  
    ecrireEcran("Entrez un nombre");  
    lireClavier(n1);  
    ecrireEcran("Entrez un nombre");  
    lireClavier(n2);  
    moyenne(entE n1, entE n2, sortE res);  
    ecrireEcran("La moyenne de ",n1," et ",n2," est ",,moy); //moy n'est pas connu à ce niveau  
fin
```

Les procédures et fonctions

Plan

- O Les procédures
 - Procédure sans paramètre
 - Procédure avec paramètres
- O Les fonctions (qui retournent des résultats)
- O Les fonctions récursives

Les procédures et fonctions

Les fonctions

- Une fonction est une procédure particulière **qui retourne un résultat**
- Une fonction ne peut retourner **qu'un seul** résultat
- Les fonctions se déclarent au même endroit que les procédures
- Les fonctions se rapprochent de la notion mathématique des fonctions
 $z = f(x,y)$
- Si une fonction n'a pas de retour, c'est une procédure

Remarque :

- Si une procédure n'a qu'un seul paramètre de sortie, il est préférable (...) et plus facile) d'utiliser une fonction

Les procédures et fonctions

Les fonctions : le pseudo-code

programme Premier **c'est**

fonction <nomDeFonction>(<liste paramètres>) **délivre** <type> **c'est**

debut

resultat : <type>

<déclaration des variables de la fonction>

<Instructions de la fonction>

retourne <resultat>;

fin

// programme principal

debut

res : <type>;

res := <nomDeFonction>(<liste paramètres>); **// appel de la fonction**

fin

- O **Bonne pratique** : déclarer une variable du type du retour, elle sera la valeur retournée.
- O l'instruction **retourne** à la dernière ligne de la fonction

Les procédures et fonctions

Exemple d'une fonction qui retourne la moyenne de deux entiers

```
programme moy2Nb c'est
```

```
// Definition de la fonction
```

```
fonction calculMoyenne(entF v1 : entier, entF v2 : entier) délivre réel c'est
```

```
  debut
```

```
    resultat : réel
```

```
    résultat:= (v1+v2)/2.0;
```

```
    retourne resultat
```

```
  fin
```

```
// Definition du programme principal
```

```
debut
```

```
  n1, n2 : entier;
```

```
  moyenne : réel;
```

```
  ecrireEcran("Entrez un nombre");
```

```
  lireClavier(n1);
```

```
  ecrireEcran("Entrez un nombre");
```

```
  lireClavier(n2);
```

```
  moyenne := calculMoyenne(entE n1,entE n2); // Appel de la fonction
```

```
  ecrireEcran("La moyenne de ",n1," et ",n2," est ",moyenne);
```

```
fin
```

Les procédures et fonctions

Plan

- O Les procédures
 - Procédure sans paramètre
 - Procédure avec paramètres
- O Les fonctions (qui retournent des résultats)
- O Les fonctions récursives

Les procédures et fonctions

Un mot sur la récursivité

- Les procédures et fonctions peuvent être appelées à partir d'autres procédures et fonctions
- Une procédure ou fonction peut s'appeler elle-même !
on parle alors de **récursivité**.

Remarques sur la récursivité :

- Très économique pour le programmeur ; elle permet de faire les choses performantes, en très peu d'instructions.
- Très coûteuse en ressources machine. La machine va être obligée de créer autant de variables temporaires que de "tours" de fonction en attente.
- La notion de récursivité n'est pas nécessaire mais indispensable à connaître.

Les procédures et fonctions

Exemple de récursivité : calcul de la factorielle d'un nombre

O $a! = a (a-1) (a-2) (a-3) \dots (1)$

O $a! = a (a-1)!$: la propriété qui va permettre la récursivité

O $1! = 1$ $0! = 1$

programme factorielleRecur **c'est**

fonction factorielle(**entE** a : entier) **délivre** entier **c'est**

debut

res : entier;

ecrireEcran("a = ", a)

si (a>0)

res := a*factorielle(**entE** (a-1)); // appel => paramètre effectif

sinon // test de fin de récursivité

res := 1;

finsi

ecrireEcran("res = ", res)

retourne res;

fin

debut

nb, resultat : entier;

nb := 5

resultat:= factorielle(entE nb)

ecrireEcran(nb,"! = ", resultat);

fin

Affichage à l'écran :

a = 5

a = 4

a = 3

a = 2

a = 1

res = 1

res = 2

res = 6

res = 24

res = 120

5 ! = 120